大数据应用人才培养系列教材

Python语言

刘 鹏　张 燕　○总主编
李肖俊 ○主编　　刘 河　钟 涛 ○副主编

清華大學出版社
TSINGHUA UNIVERSITY PRESS

第十一章　数据分析

11.1 数据存储

11.2 数据清洗

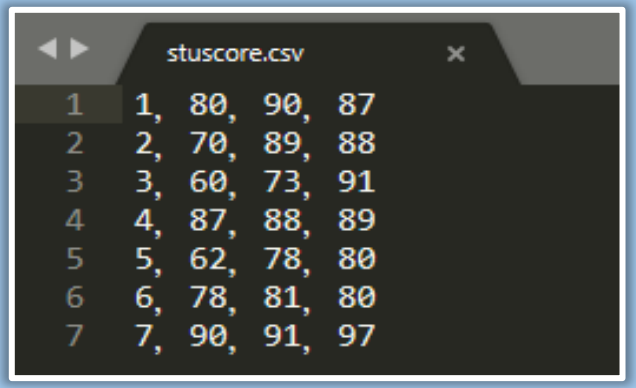
11.3 NumPy

CSV文件存取

数据存取是数据分析的基础，尤其是面对海量数据，数据的存取方式显得尤为重要。本节，以Pandas库对象为基础重点介绍python数据分析中常见的几种数据存取方法。

CSV（Comma-Separated Value，逗号分隔值）是一种常见的文件格式。通常，数据库的转存文件就是CSV格式的，文件中的各个字段对应于数据库表中的列。 在Pandas中我们可以使用read_csv()函数将.csv 数据读入程序。

比如，读取学生成绩数据，首先创建一个stuscore.csv 文件，然后使用pandas对象的read_csv()函数读取并显示数据。



CSV文件存取

first.py

```python
import pandas as pd
import numpy as np
data = pd.read_csv('./stuscore.csv')
print(data)
```

REPL [python]

```
   1, 80, 90, 87
0  2, 70, 89, 88
1  3, 60, 73, 91
2  4, 87, 88, 89
3  5, 62, 78, 80
4  6, 78, 81, 80
5  7, 90, 91, 97

***Repl Closed***
```

CSV文件存取

```
>>> import  pandas as pd
>>> import  numpy as np
>>> import  matplotlib.pyplot as plt
>>> names = ['张三','李四','王五','赵六','郭七']
>>> ages = [20,19,23,18,19]
>>> DataSet = list(zip(names,ages))
>>> DataSet
[('张三', 20), ('李四', 19), ('王五', 23), ('赵六', 18), ('郭七', 19)]
>>> df = pd.DataFrame(data = DataSet ,columns =['Name','Age'])
>>> df   #输出pandas对象创建的数据表
  Name  Age
0  张三   20
1  李四   19
2  王五   23
3  赵六   18
4  郭七   19
>>> df.to_csv('./info.csv',index = False,header = False) #将生成的数据df写入info.cvs
#然后，读取info.csv 文件，用于验证info.csv文件创建成功，示例代码如下：
>>> data = pd.read_csv('./info.csv') #读取info.csv中的数据以验证写入成功
>>> print(data)
0  张三  20
1  李四  19
2  王五  23
3  赵六  18
4  郭七  19
>>>
```

XLSX文件的存取

使用pandas 读取Excel电子表格中的数据，需借助第三方库xlrd 完成Excel表数据的读写操作。用read_excel() 函数完成Excel电子表格中数据的读取，用to_excel()函数完成数据pandas DataFrame 中的数据写入Excel。

为了完成Excel电子表格中数据的存取，我们先来完成Python第三方库openpyxl、xlsxwriter、xlrd、的安装。这里，还是使用 pip install命令完成。

示例Excel 数据 图13.6：

	A	B	C	D
1	name	class	score	
2	张三	三一班	370	
3	李四	三二班	500	
4	王五	三三班	600	
5	赵六	三二班	580	
6	郭七	三三班	700	
7				

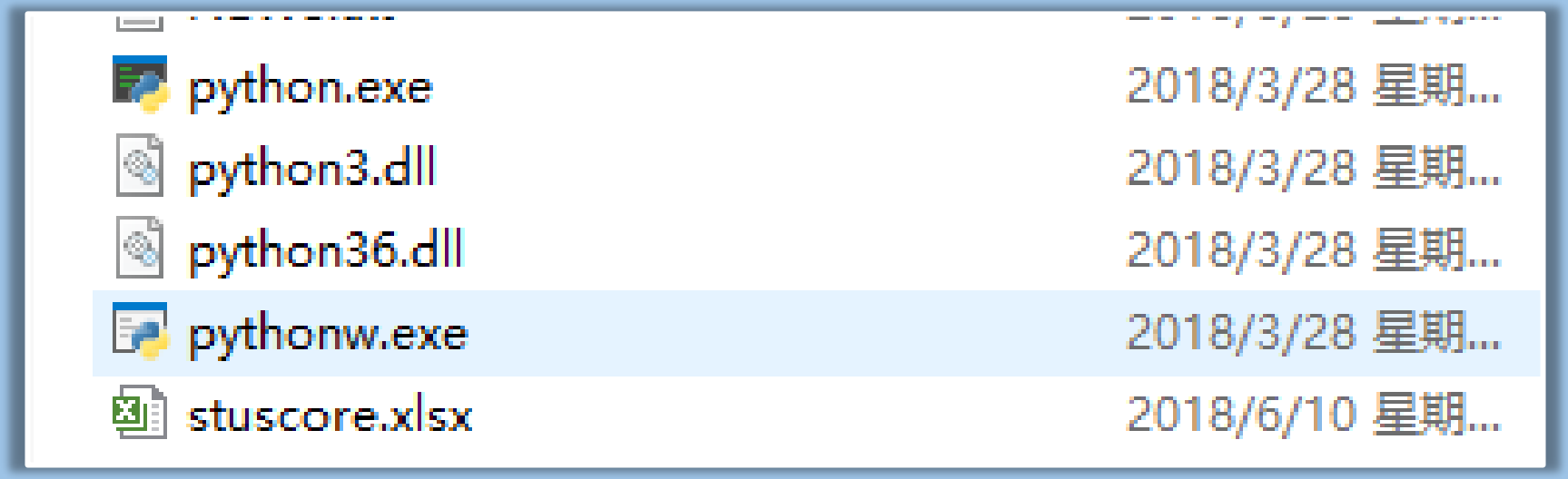
XLSX文件的存取

首先，我们用pandas的数据框来创建如图13.6的Excel 数据表，然后，将创建的Excel表的数据输出。

```
>>> import pandas as pd
>>> df_out = pd.DataFrame([('张三', '三一班',370),('李四', '三二班',500), ('王五','三三班',600),('赵六', '三二班',580),('郭七', '三三班',700)], columns=['name', 'class','score'])
>>> df_out.to_excel('stuscore.xlsx') # 生成 excel 文件
>>> pd.read_excel('stuscore.xlsx') #读取 excel 文件
   name  class   score
0   张三   三一班    370
1   李四   三二班    500
2   王五   三三班    600
3   赵六   三二班    580
4   郭七   三三班    700
```

XLSX文件的存取

df_out写入的excel文件通常位于Python的安装目录下，如下图所示：



第十三章　项目实战：数据分析

11.1 数据存储

11.2 数据清洗

11.3 NumPy

简介

大数据时代，数据便是掘金的黄金地带。企业大量的历史数据能否发挥其应有的价值，取决于企业采用什么样的分析手段，去发掘数据本身所蕴含的规律。数据分析人次炙手可热，已成为大数据时代企业争抢的焦点。本章，将以Python 技术为基础，通过实际案例的讲解来使大家对数据分析的流程达到定性的认识。同时，通过课程实验，提高大家的动手能力，为使大家成为数据分析人才做好启蒙教育。

编码问题

通常，源数据分布在不同的业务流程之中。而不同的业务流程中对数据的要求、理解和规格各不相同。导致对同一数据对象的描述千差万别。因此，在清洗数据的过程中，首先要对数据的编码格式做统一要求。

对于数据项的约定可从以下几个方面进行：

◎命名规则：对于同一数据对象，其名称应当是唯一的。比如页面访问量这个字段，可能称作访问深度、分为PV数、页面浏览量等。

◎数据类型：同一个数据对象的类型必须一致，而且表示方法唯一，如普通日期和时间戳的区分。

◎计数方法：对于数值类型的数据，单位务必统一。如：重量单位，千克、公斤、克、斤等，在数据表中必须用唯一单位。

◎约束条件：数据表之间的关系约定不能产生二义性。比如：表的主键、唯一性、外键约束等。

缺失值分析

数据的缺失，主要包括记录的缺失和记录中某各字段信息的缺失。两者都会造成最终分析结果的不准确。下面，从缺失值产生的原因及处理方法进行介绍。

1 产生原因

缺失值产生的原因主要包括三大类，具体如下：

①出于信息安全的需求。由于某种原因无法获取，或者获取成本过高。

②人为的信息遗漏。可能是由于个人主观认识不到位，导致的因人为因素产生的遗漏。也可能是由于数据获取设备的故障所引起的非人为原因产生的丢失。

③字段值的缺失。某些情况下，缺失值不一定意味者着数据的错误。比如儿童的手机号码、个人收入等字段值。

2 数据值的缺失，通常会给数据分析带来如下影响：

①数据挖掘建模将丢失大量的有用信息。

缺失值分析

②数据挖掘模型表现出来的不确定性更加显著，数据背后蕴含的规律更难发掘。

③字段的空值会导致数据分析过程陷入混乱，致使分析产生不可靠的结果。

3 应对策略

生活中我们所采集到的数据常错综复杂，其值的缺失也是很常见。那么我们该如何处理这些缺失值呢？常用的有三大类方法，即删除法、填补法和插值法。

◎删除法：当数据中的某个变量大部分值都是缺失值，可以考虑删除改变量；当缺失值是随机分布的，且缺失的数量并不是很多是，也可以删除这些缺失的观测

◎替补法：对于连续型变量，如果变量的分布近似或就是正态分布的话，可以用均值替代那些缺失值；如果变量是有偏的，可以使用中位数来代替那些缺失值；对于离散型变量，我们一般用众数去替换那些存在缺失的观测。

◎插补法：插补法是基于蒙特卡洛模拟法，结合线性模型、广义线性模型、决策树等方法计算出来的预测值替换缺失值。

去除异常值

异常值，是指数据样本中的个别值，其数值明显偏离对应字段的所有观察值。异常值又称离群点。异常值的分析是检验数据集中是否存在录入错误以及不合常理的数据。去除异常值的方法主要包括：

◎统计分析法

通常对变量的取值做一个简单的量化统计，尤其是数值型字段。进而查看那些取值超出合法取值范围。最常用的统计方法是求最大值、平均值、最小值。用最小值和最大值确定正常取值范围。用平均值替代空白字段值，将超出合理取值的记录剔除采样数据。比如：个人信息中的年龄字段取值超过150 就属于异常取值，可考虑用平均取值替代。

◎ 3δ分析法

通常，如果数据服从正态分布，在3δ思想的指导下，异常值被认定为与平均值偏差超过3倍标准差的数值。因为，在正态分布下，距离大于3倍标准差的数值的概率小与等于0.003，属于小概率事件。相反，若数据字段值不服从正态分布，可用远离平均值多少倍标准差约定异常数值。

去除重复值与冗余信息

由于各种各样的原因，在获取的数据源中，经常存在重复的字段、重复的记录以及获取了与分析主题无关的数据项。这时，为了提高数据的质量，我们需要对源数据做去重处理和冗余处理。

对于重复数据的处理，我们通常采用的方法是“排序合并”。具体做法是：现将数据库表中的记录按照指定的规则排序，然后通过比较邻近记录是否相似来检测记录是否有重复。这项工作包括排序和相似度计算两个步骤。常用的排序方法有：插入排序、冒泡排序、快速排序、希尔排序等。常用的相似度计算方法有：基本的字段匹配算法、标准的欧氏距离法、相关系数、信息熵等。

另外，需要注意的是，对重复的数据项，尽量通过具体分析主题确定相关提取规则。在数据清洗阶段，对重复的数据切勿轻易的进行删除。尤其是不能将与分析主题相关的重要业务数据过滤掉。

对于与分析主题无关的数据项，也即我们通常说的冗余信息，同样，也不可直接剔除出数据源。而需要根据制定的提取规则通过子表的形式，生成新的和分析主题相关的数据表。

数据清洗实例

isnull() 判断单元格是否无效

```
import pandas as pd

data=pd.DataFrame([['张三',22,400],['李四','',401],['王二','',488],['马五',89,500]],columns=['名字','年龄','成绩'])
data.to_excel('data.xlsx',index=False)
data1=pd.read_excel('data.xlsx')
print(data1['年龄'])
print(data1['年龄'].isnull())
```

```
0    22.0
1     NaN
2     NaN
3    89.0
Name: 年龄, dtype: float64
0    False
1     True
2     True
3    False
```

数据清洗实例

自定义无效单元格

```
import pandas as pd

data=pd.DataFrame([['张三',22,'-'],['李四','',401],['王二','',488],['123',89,500]],columns=['名字','年龄','成绩'])
data.to_excel('data.xlsx',index=False)
data1=pd.read_excel('data.xlsx',na_values=['-','123'])
print(data1)
print(data1.isnull())
```

```
    名字    年龄     成绩
0   张三  22.0    NaN
1   李四   NaN  401.0
2   王二   NaN  488.0
3  NaN  89.0  500.0
      名字     年龄     成绩
0  False  False   True
1  False   True  False
2  False   True  False
3   True  False  False
```

数据清洗实例

dropna(): 删除无效数据

```
import pandas as pd

data=pd.DataFrame([['张三',22,'-'],['李四','',401],['123','',488],['马五',89,500]],columns=['名字','年龄','成绩'])
data.to_excel('data.xlsx',index=False)
data1=pd.read_excel('data.xlsx',na_values=['-','123'])
new_data=data1.dropna()
print(new_data)
```

```
   名字   年龄    成绩
3  马五  89.0  500.0
```

数据清洗实例

dropna(inplace=True): 删除无效数据

```
import pandas as pd

data=pd.DataFrame([['张三',22,'-'],['李四','',401],['123','',488],['马五',89,500]],columns=['名字','年龄','成绩'])
data.to_excel('data.xlsx',index=False)
data1=pd.read_excel('data.xlsx',na_values=['-','123'])
data1.dropna(inplace=True)
print(data1)
```

```
   名字    年龄     成绩
3  马五  89.0  500.0
```

数据清洗实例

dropna(subset=[], inplace=True): 删除部分无效数据

```
import pandas as pd

data=pd.DataFrame([['张三',22,'-'],['李四','',401],['123','',488],['马五',89,500]],columns=['名字','年龄','成绩'])
data.to_excel('data.xlsx',index=False)
data1=pd.read_excel('data.xlsx',na_values=['-','123'])
data1.dropna(subset=['名字'],inplace=True)
print(data1)
```

```
   名字    年龄     成绩
0  张三  22.0    NaN
1  李四   NaN  401.0
3  马五  89.0  500.0
```

数据清洗实例

fillna(data, inplace=True): 替换无效数据

```
import pandas as pd

data=pd.DataFrame([['张三',22,'-'],['李四','',401],['123','',488],['马五',89,500]],columns=['名字','年龄','成绩'])
data.to_excel('data.xlsx',index=False)
data1=pd.read_excel('data.xlsx',na_values=['-','123'])
data1.fillna(12345,inplace=True)
print(data1)
```

```
      名字       年龄       成绩
0     张三     22.0  12345.0
1     李四  12345.0    401.0
2  12345  12345.0    488.0
3     马五     89.0    500.0
```

数据清洗实例

fillna(data, inplace=True): 替换部分无效数据

```
import pandas as pd

data=pd.DataFrame([['张三',22,'-'],['李四','',401],['123','',488],['马五',89,500]],columns=['名字','年龄','成绩'])
data.to_excel('data.xlsx',index=False)
data1=pd.read_excel('data.xlsx',na_values=['-','123'])
data1['年龄'].fillna(12345,inplace=True)
print(data1)
```

```
     名字       年龄     成绩
0    张三     22.0    NaN
1    李四  12345.0  401.0
2   NaN  12345.0  488.0
3    马五     89.0  500.0
```

数据清洗实例

mean(), median(), mode(): 计算均值，中位数，众数

```
import pandas as pd

data=pd.DataFrame([['张三',26,'-'],['李四',24,401],['王二',24,488],['马五',25,500]],columns=['名字','年龄','成绩'])
data.to_excel('data.xlsx',index=False)
data1=pd.read_excel('data.xlsx',na_values=['-','123'])
data1.dropna(inplace=True)
print(data1['年龄'].mean(),data1['年龄'].median(),data1['年龄'].mode())
```

```
24.333333333333332 24.0 0    24
dtype: int64
```

*经常会以平均值来填充无效值

数据清洗实例

Data1.loc(index,column)=correct_result: 修改错误数据

```
import pandas as pd

data=pd.DataFrame([['张三',25,'-'],['李四',24,401],['王二',24,488],['马五',1000,500]],columns=['名字','年龄','成绩'])
data.to_excel('data.xlsx',index=False)
data1=pd.read_excel('data.xlsx',na_values=['-','123'])
data1.loc[3,'年龄']=100
print(data1)
```

	名字	年龄	成绩
0	张三	25	NaN
1	李四	24	401.0
2	王二	24	488.0
3	马五	100	500.0

数据清洗实例

Data1.loc(index,column)=correct_result: 设置条件语句修改错误数据

```
import pandas as pd

data=pd.DataFrame([['张三',26,'-'],['李四',24,401],['王二',24,488],['马五',1000,500]],columns=['名字','年龄','成绩'])
data.to_excel('data.xlsx',index=False)
data1=pd.read_excel('data.xlsx',na_values=['-','123'])
for i in data1.index:
    if data1.loc[i,'年龄']>25:
        data1.loc[i, '年龄']=25
print(data1)
```

	名字	年龄	成绩
0	张三	25	NaN
1	李四	24	401.0
2	王二	24	488.0
3	马五	25	500.0

数据清洗实例

drop(index): 设置条件语句删除部分数据

```
import pandas as pd

data=pd.DataFrame([['张三',26,'-'],['李四',24,401],['王二',24,488],['马五',1000,500]],columns=['名字','年龄','成绩'])
data.to_excel('data.xlsx',index=False)
data1=pd.read_excel('data.xlsx',na_values=['-','123'])
data1.dropna(inplace=True)
for i in data1.index:
    if data1.loc[i,'年龄']>25:
        data1.drop(i,inplace=True)
print(data1)
```

```
   名字  年龄     成绩
1  李四  24  401.0
2  王二  24  488.0
```

数据清洗实例

duplicated(): 查询重复数据

```
import pandas as pd

data=pd.DataFrame([['李四',24,401],['李四',24,401],['王二',24,488],['马五',1000,500]],columns=['名字','年龄','成绩'])
data.to_excel('data.xlsx',index=False)
data1=pd.read_excel('data.xlsx',na_values=['-','123'])
print(data1.duplicated())
```

```
0    False
1     True
2    False
3    False
dtype: bool
```

数据清洗实例

drop_duplicates(): 删除重复数据

```
import pandas as pd

data=pd.DataFrame([['李四',24,401],['李四',24,401],['王二',24,488],['马五',1000,500]],columns=['名字','年龄','成绩'])
data.to_excel('data.xlsx',index=False)
data1=pd.read_excel('data.xlsx',na_values=['-','123'])
data1.drop_duplicates(inplace=True)
print(data1)
```

	名字	年龄	成绩
0	李四	24	401
2	王二	24	488
3	马五	1000	500

数据清洗实例

drop_duplicates(subset): 删除部分重复数据

```
import pandas as pd

data=pd.DataFrame([['李四',23,401],['李四',24,401],['王二',24,488],['马五',1000,500]],columns=['名字','年龄','成绩'])
data.to_excel('data.xlsx',index=False)
data1=pd.read_excel('data.xlsx',na_values=['-','123'])
data1.drop_duplicates(subset=['名字'],inplace=True)
print(data1)
```

	名字	年龄	成绩
0	李四	23	401
2	王二	24	488
3	马五	1000	500

第十一章　项目实战：爬虫程序

11.1 数据存储

11.2 数据清洗

11.3 NumPy

简介

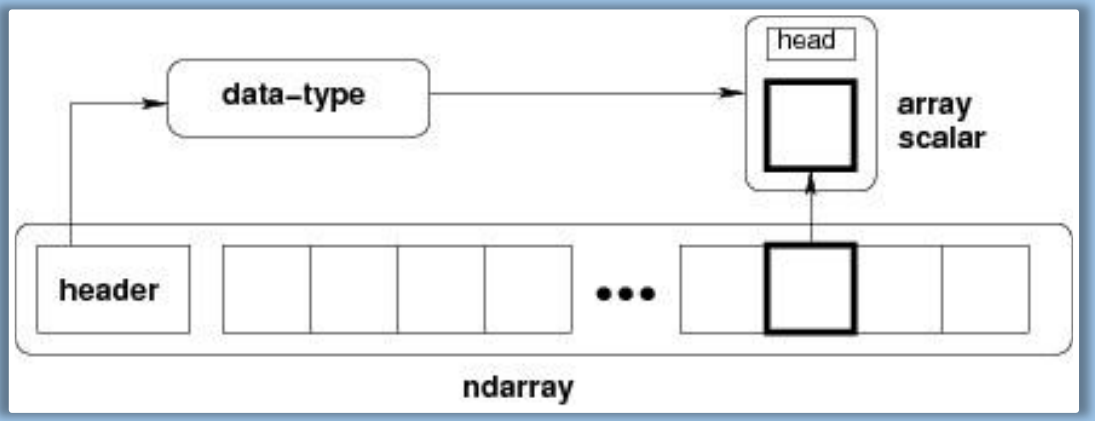
NumPy（Numerical Python的缩写）是一个开源的Python科学计算库。使用NumPy，就可以很方便地使用数组和矩阵。 NumPy包含很多实用的数学函数，涵盖线性代数运算、傅里叶变换和随机数生成等功能。

NumPy已成为Python科学计算生态系统的重要组成部分，其在保留Python语言优势的同时大大增强了科学计算和数据处理的能力。更重要的是， NumPy与SciPy、 Matplotlib等其它众多Python科学计算库很好地结合在一起，共同构建了一个完整的科学计算生态系统。一句话， NumPy是使用Python进行数据分析的一个必备工具。

基本操作

Numpy 中的ndarray 是一个多维数组对象，该对象由描述数据的元数据和数据本身两部分组成。通常，大部分的操作仅仅是针对修改描述数据的元数据部分，而不改变实际数据本身。Numpy中的数组一般是同质的，亦即数据的类型是一致的，这样的规定最大的好处在于方便估算数组所需的存储空间。

与Python 类似，Numpy数组的下标也是从零开始的。ndarray中的每个元素在内存中使用相同大小的块。 ndarray中的每个元素是数据类型对象的对象(称为dtype)。从ndarray对象提取的任何元素(通过切片)由一个数组标量类型的 Python 对象表示。



基本操作

通常，ndarray对象是使用 NumPy 中的数组函数创建的，定义如下：

numpy.array(object, dtype = None, copy = True, order = None, subok = False, ndmin = 0) 其参数的含义如下表。

参数	含义
object	任何暴露数组接口方法的对象都会返回一个数组或任何(嵌套)序列。
dtype	数组的所需数据类型，可选。
copy	可选，默认为 true，对象是否被复制。
order	C(按行)、F(按列)或 A(任意，默认)。
subok	默认情况下，返回的数组被强制为基类数组。 如果为 true，则返回子类。
ndimin	指定返回数组的最小维数。

基本操作

创建数组最简单的办法就是使用array函数，我们可以通过array函数传递Python的序列对象创建数组。

```
In [1]: import numpy as np

In [2]: a = np.array([1,2,3,4,5])

In [3]: a
Out[3]: array([1, 2, 3, 4, 5])

In [4]: b = np.array([[1,2,3],[4,5,6],[7,8,9,]])

In [5]: b
Out[5]:
array([[1, 2, 3],
       [4, 5, 6],
       [7, 8, 9]])
```

基本操作

数组的元素类型可以通过dtype属性获得，我们可以通过dtype参数在创建时指定元素类型。

```
In [6]: c = np.array([[1,2,3,4,5,],[6,7,8,9,10],[11,12,13,14,15]],dtype = np.fl
   ...: oat)

In [7]: c
Out[7]:
array([[ 1.,  2.,  3.,  4.,  5.],
       [ 6.,  7.,  8.,  9., 10.],
       [11., 12., 13., 14., 15.]])

In [8]: d = np.array([[16,17,18],[19,20,21],[22,23,24],[25,26,27]],dtype = np.complex)

In [9]: d
Out[9]:
array([[16.+0.j, 17.+0.j, 18.+0.j],
       [19.+0.j, 20.+0.j, 21.+0.j],
       [22.+0.j, 23.+0.j, 24.+0.j],
       [25.+0.j, 26.+0.j, 27.+0.j]])
```

基本操作

然而，通过创建Python序列，借助array函数将序列转换为数组效率不高。为此Numpy专门提供了很多用来创建数组的内置函数。举例如下：

arange()函数: 通过指定开始值、终止值和步长来创建一维数组,数组不包含终止值；

linspace()函数:通过指定开始值、终值和元素个数来创建一维数组，可以通过endpoint关键字指定是否包括终值，缺省设置是包括终值;

logspace()函数和linspace()函数类似，不过它创建等比数列。

基本操作

```
In [10]: e = np.arange(1,20,3)

In [11]: e
Out[11]: array([ 1,  4,  7, 10, 13, 16, 19])

In [12]: f = np.linspace(1,20,30)

In [13]: f
Out[13]:
array([ 1.        ,  1.65517241,  2.31034483,  2.96551724,  3.62068966,
        4.27586207,  4.93103448,  5.5862069 ,  6.24137931,  6.89655172,
        7.55172414,  8.20689655,  8.86206897,  9.51724138, 10.17241379,
       10.82758621, 11.48275862, 12.13793103, 12.79310345, 13.44827586,
       14.10344828, 14.75862069, 15.4137931 , 16.06896552, 16.72413793,
       17.37931034, 18.03448276, 18.68965517, 19.34482759, 20.        ])

In [14]: g = np.logspace(1,20,10)

In [15]: g
Out[15]:
array([1.00000000e+01, 1.29154967e+03, 1.66810054e+05, 2.15443469e+07,
       2.78255940e+09, 3.59381366e+11, 4.64158883e+13, 5.99484250e+15,
       7.74263683e+17, 1.00000000e+20])
```

基本操作

多维数组：

多维数组有多个轴，因此它的下标需要用多个值来表示，NumPy采用组元(tuple)作为数组的下标。其基本属性包括：

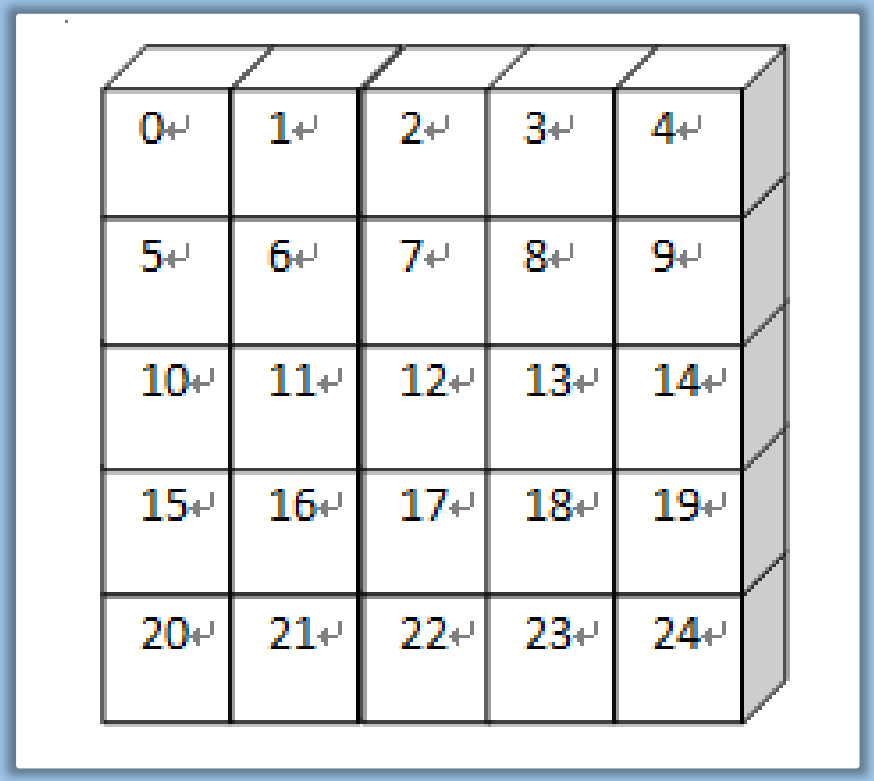
轴（axis）：每一个线性的数组称为是一个轴，也就是维度（dimensions），用ndarray.shape 表示。比如，二维数组相当于两个一维数组的嵌套，其中第一个一维数组中每个元素又是一个一维数组，因而，一维数组就是ndarray中的轴，第一个轴（也就是第0轴）相当于一个容器数组，第二个轴（也就是第1轴）是容器数组里的数组。

秩（rank）：维数，一维数组的秩为1，二维数组的秩为2，以此类推，即轴的个数。用ndarray.ndim表示。

如下图所示，a为一个5x5的数组逻辑结构示意图，可以看成由5个一维数组构成，每个一维数组包含5个元素；第一维被称为第0轴（列），第二维被称为第1轴（行）；秩为2，维度为（5，5），元素总个数为25。

基本操作

5*5 的二维数组



基本操作

代码实现：

```
Python 3.6.5 (v3.6.5:f59c0932b4, Mar 28 2018, 17:00:18) [MSC v.1900 64 bit (AMD64)] on win32
Type "copyright", "credits" or "license()" for more information.
>>> import numpy as np
>>> h = np.array([(0,1,2,3,4),(5,6,7,8,9),(10,11,12,13,14),(15,16,17,18,19),(20,21,22,23,24)])
>>> h
array([[ 0,  1,  2,  3,  4],
       [ 5,  6,  7,  8,  9],
       [10, 11, 12, 13, 14],
       [15, 16, 17, 18, 19],
       [20, 21, 22, 23, 24]])
>>> h.ndim
2
>>> h.shape
(5, 5)
>>> h.size
25
>>> h.dtype
dtype('int32')
>>> h.itemsize
4
>>>
```

基本操作

Numpy 中数组的存取方法和 Python 标准的方法相同。其存取方式依次介绍如下：

◎对于一维数组，其操作类似于 Python 中的 list。用整数作为下标可以获取数组中的某个元素，比如：

```
>>> a = np.arange(10)
>>> a[7]
7
>>>
```

用范围作为下标获取数组的一个切片，但不包括起始元素和终止元素，相当于数组的子集。比如：

```
>>> a[3:5]
array([3, 4])
>>>
```

在获取指定数组的一个切片时，可以省略起始位置或结束位置。默认表示从第零个元素开始或直至最后一个元素。比如：

```
>>> a[:7]
array([0, 1, 2, 3, 4, 5, 6])
>>> a[3:]
array([3, 4, 5, 6, 7, 8, 9])
>>>
```

基本操作

多维数组的存取，以二维为例。

```
>>> import numpy as np
>>> h = np.array([(0,1,2,3,4),(5,6,7,8,9),(10,11,12,13,14),(15,16,17,18,19),(20,21,22,23,24)])

>>> h
array([[ 0,  1,  2,  3,  4],
       [ 5,  6,  7,  8,  9],
       [10, 11, 12, 13, 14],
       [15, 16, 17, 18, 19],
       [20, 21, 22, 23, 24]])
>>> h[1,3:5]
array([8, 9])
>>> h[3:,3:]
array([[18, 19],
       [23, 24]])
>>> h[:,3]
array([ 3,  8, 13, 18, 23])
>>> h[ :,:]
array([[ 0,  1,  2,  3,  4],
       [ 5,  6,  7,  8,  9],
       [10, 11, 12, 13, 14],
       [15, 16, 17, 18, 19],
       [20, 21, 22, 23, 24]])
>>>
```

感谢聆听